

EECE 7205: Computing Fundamentals for Engineers

Fall 2026

Lab Assignment #01 Algorithm Analysis Workflow

Topic: Algorithm Analysis

Student Name	Eduardo Scopelli
NUID	003972121
Email	scopellirivas.e@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	09-20-2026

Checklist Item

Goal: For each problem, complete the following steps in order. Your report and C++ program must show that you followed the same process used in class.

- **Understand the problem:** Identify the goal, inputs, outputs, constraints, assumptions, and important edge cases.
- **Design the solution:** Select an appropriate algorithm or data structure and briefly explain why it fits the problem.
- **Write pseudocode:** Write clear, language-independent pseudocode before writing the C++ implementation. Use the same pseudocode format and level of detail demonstrated in class.
- **Explain correctness:** Briefly explain how the algorithm produces the required result and handles duplicates, ties, empty inputs, and other relevant edge cases.
- **Analyze complexity:** Determine and justify the time complexity of your algorithm using Big-O notation. Include best-case, average-case, and worst-case complexity when they differ. Also state the space complexity.
- **Implement in C++:** Translate the pseudocode into a clear, functional C++ solution. Use meaningful names, functions, consistent formatting, and comments for non-obvious logic.
- **Test the solution:** Use the provided example and at least two additional test cases for each problem, including at least one edge case. Include screenshots that clearly show the selected problem, input, and output.

Problem 1

Description: Intervals appear in calendars, resource reservations, timelines, and scheduling systems. Overlapping intervals can be combined into a smaller set of non-overlapping intervals.

Question: Given an array of intervals [start, end], sort the intervals by starting value and merge every overlapping interval. Return the resulting non-overlapping intervals.

Understand the problem

- **Goal:** Sort array by start intervals, then merge based on overlapping end intervals.
- **Inputs:** Array of Intervals [start, end]
- **Outputs:** Array of sorted, non-overlapping Intervals [start, end]
- **Constraints:** end > start
- **Assumptions:** Interval values represent hours, touching intervals (eg. [1,2],[2,3]) also get merged, start ≥ 0 , end ≤ 24 , .
- **Edge cases:** Less than 2 intervals: no merge required.

Design the solution

The problem provides a 2D array as an input and requires the same data structure for the output, so no changes in the data structure are required.

As for the logic, the array of intervals must first be sorted by the start times of each interval. This can be accomplished through a simple bubble sort based only on the start values (ignoring overlap and duplicates) which would iterate over every interval, compare its start time to the next interval, and swap them in case the previous interval starts after the next. With the array sorted, overlapping intervals must be merged, considering two main cases: Fully nested intervals and partially overlapping intervals. In the case of fully nested intervals, only the larger interval is maintained and the smaller one is eliminated. In the case of partially overlapping intervals, the smallest start time and largest end time create the new merged interval. In order to be able to iterate over all original intervals, the iterative loop should start on the second element of the array and compare it with the previous element, ensuring that only the previous iteration is affected / deleted without affecting the order of iterations.

Write pseudocode

```
1 n ← SizeOf(Intervals)
2
3 if n <← 1 then
4     return Intervals
5 end if
6
7 for Interval ← 0 to n - 2 do
8     for i ← 0 to n - Interval - 2 do
```

```

9      if Intervals[i].start > Intervals[i + 1].start then
10         swap(Intervals[i], Intervals[i + 1])
11     end if
12 end for
13 end for
14
15 merged ← []
16 prevI ← Intervals[0]
17 for i ← 1 to n - 1 do
18     if Intervals[i].start <← prevI.end then
19         if Intervals[i].end > prevI.end then
20             prevI.end ← Intervals[i].end
21         end if
22     else
23         append prevI to merged
24         prevI ← Intervals[i]
25     end if
26 end for
27 return merged

```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Intervals})$	1
Check $n \leq 1$	1
return Interval	1
Initialize $\text{Interval} \leftarrow 0$	1
Check $\text{Interval} < n - 2$	$n - 1$
Increment Interval	$n - 2$
Initialize $i \leftarrow 0$	$n - 2$
Check $i < n - \text{Interval} - 2$	$\frac{(n-1)n}{2}$
Increment i	$\frac{(n-2)(n-1)}{2}$
Check $\text{Intervals}[i].\text{start} > \text{Intervals}[i + 1].\text{start}$	$\frac{(n-2)(n-1)}{2}$
Swap $\text{Intervals}[i].\text{start}$ and $\text{Intervals}[i + 1].\text{start}$	$\frac{(n-2)(n-1)}{2}$
$\text{merged} \leftarrow []$	1
$\text{prevI} \leftarrow \text{Intervals}[0]$	1
Initialize $i \leftarrow 1$	1
Check $i < n - \text{Interval} - 2$	$n - 1$
Increment i	$n - 2$
Check $\text{Intervals}[i].\text{start} < \text{prevI}.end$	$n - 2$
Check $\text{Intervals}[i].end > \text{prevI}.end$	$n - 2$
$\text{prevI}.end \leftarrow \text{Intervals}[i].end$	$n - 2$
Else statement	0
Append prevI to merged	$n - 2$
$\text{prevI} \leftarrow \text{Intervals}[i]$	$n - 2$
Return merged	1

Time complexity (Best case): $3 \rightarrow O(1)$

Time complexity (Worst case): $2n^2 + 6n - 6 \rightarrow O(n^2)$

Implement in C++

```
// P1
#pragma region
vector<vector<int>> case1_1 = {{1,3}, {15,18}, {8,10}, {2,6}};
vector<vector<int>> ans1_1 = {{1,6}, {8,10}, {15,18}};
assert(mergeIntervals(case1_1) == ans1_1);

vector<vector<int>> case1_2 = {{1,7}, {7,10}, {2,4}};
vector<vector<int>> ans1_2 = {{1,10}};
assert(mergeIntervals(case1_2) == ans1_2);

vector<vector<int>> case1_3 = {{1,5}};
vector<vector<int>> ans1_3 = {{1,5}};
assert(mergeIntervals(case1_3) == ans1_3);

cout << "Problem 1 Test Cases: Passed" << endl;
#pragma endregion

(.venv) scopellie@DESKTOP-0W0ARFH:~/EECE7205$ /home/scopellie/EECE7205/Assignment01_Scopelli
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

Problem 2

Description: A manufacturing line labels items with three categories represented by 0, 1, and 2. The items must be grouped in category order.

Question: Given an array containing only 0, 1, and 2, sort it in place. First implement a counting-based solution. Then implement the one-pass Dutch National Flag algorithm without using `std::sort()`.

Understand the problem

- **Goal:** Sort array containing only 0, 1 and 2 categories by a counting method, and by the one-pass Dutch National Flag algorithm.
- **Inputs:** Array containing 0, 1 and 2.
- **Outputs:** Sorted array.
- **Constraints:** Values are either 0, 1 or 2.
- **Assumptions:**
- **Edge cases:**

0.1 Case 1. Count-based solution

Design the solution

The problem provides an array of integers/categories with the need to sort this array, for which the data structure can be maintained.

In order to solve the problem through a counting method, we have to establish a counter per class (0, 1 and 2), iterate over the original array to establish the frequency of each class, and create a new array in which we will push each class in order their corresponding amount of times based on the counters.

Write pseudocode

```
1 n ← SizeOf(Labels)
2 count0 ← 0
3 count1 ← 0
4 count2 ← 0
5 for i ← 0 to n - 1 do
6     if Labels[i] = 0 then
7         count0 ← count0 + 1
8     else if Labels[i] = 1 then
9         count1 ← count1 + 1
10    else
11        count2 ← count2 + 1
12    end if
13 end for
14
15 sorted ← []
16 for i ← 0 to n - 1 do
17     if i < count0 then
18         append 0 to sorted
19     else if i < count0 + count1
20         append 1 to sorted
21     else
22         append 2 to sorted
23     end if
24 end for
25 return sorted
```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Labels})$	1
$\text{count0} \leftarrow 0$	1
$\text{count1} \leftarrow 0$	1
$\text{count2} \leftarrow 0$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Check $\text{Labels}[i] == 0$	$n - 1$
Increment count0	$n - 1$
Check $\text{Labels}[i] == 1$	$n - 1$
Increment count1	$n - 1$
Else statement	0
Increment count2	$n - 1$
$\text{sorted} \leftarrow []$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Check $i < \text{count0}$	$n - 1$
Append 0 to sorted	$n - 1$
Check $i < \text{count0} + \text{count1}$	$n - 1$
Append 1 to sorted	$n - 1$
Else statement	0
Append 2 to sorted	$n - 1$
Return merged	1

Time complexity (Best case): $8n + 2 \rightarrow O(n)$

Time complexity (Worst case): $10n \rightarrow O(n)$

Implement in C++

```
// P2 - Counter
#pragma region
vector<int> case2_1 = {0,1,2,1,2,0,0,1,2,0};
vector<int> ans2_1 = {0,0,0,0,1,1,1,2,2,2};
assert(sortCategoriesCount(case2_1) == ans2_1);

vector<int> case2_2 = {1};
vector<int> ans2_2 = {1};
assert(sortCategoriesCount(case2_2) == ans2_2);
```

```
● (.venv) scopellie@DESKTOP-6WDARFH:~/EECE7205$ /home/scopellie/EECE7205/Assignment01_Scopellie
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

0.2 Case 2. One-pass Dutch National Flag

Design the solution

The problem provides an array of integers/categories with the need to sort said array, for which the data structure can be maintained.

In this case the One-pass Dutch National Flag algorithm will be used to solve the problem.

Write pseudocode

```
1 n ← SizeOf(Labels)
2 low ← 0
3 mid ← 0
4 high ← n - 1
5 for i ← 0 to n - 1 do
6   if Labels[mid] = 0 then
7     swap(Labels[mid], Labels[low])
8     low ← low + 1
9     mid ← mid + 1
10  else if Labels[mid] = 1
11    mid ← mid + 1
12  else
13    swap(Labels[mid], Labels[high])
14    high ← high - 1
15  end if
16 end for
17 return Labels
```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Labels})$	1
$low \leftarrow 0$	1
$mid \leftarrow 0$	1
$high \leftarrow n - 1$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Check $\text{Labels}[mid] == 0$	$n - 1$
Swap $\text{Labels}[mid]$ and $\text{Labels}[low]$	$n - 1$
Increment low	$n - 1$
Increment mid	$n - 1$
Check $\text{Labels}[mid] == 1$	$n - 1$
Increment mid	$n - 1$
Else statement	0
Swap $\text{Labels}[mid]$ and $\text{Labels}[high]$	$n - 1$
Decrement $high$	$n - 1$
Return Labels	1

Time complexity (Best case): $6n + 1 \rightarrow O(n)$
Time complexity (Worst case): $5n + 2 \rightarrow O(n)$

Implement in C++

```
// P2 - Dutch National Flag
vector<int> case2_3 = {0,1,2,1,2,0,0,1,2,0};
vector<int> ans2_3 = {0,0,0,0,1,1,1,2,2,2};
assert(sortCategoriesDutch(case2_3) == ans2_3);

vector<int> case2_4 = {1};
vector<int> ans2_4 = {1};
assert(sortCategoriesDutch(case2_4) == ans2_4);
```

```
(.venv) scopellie@DESKTOP-0PQWAFH:~/EECE7205$ /home/scopellie/EECE7205/Assignment01_Scopellie
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

Problem 3

Description: A custom ordering is sometimes required when the normal numeric order does not produce the desired combined result.

Question: Given a vector of nonnegative integers, arrange them so that their concatenation forms the largest possible number. Return the answer as a string.

Understand the problem

- **Goal:** Sort array in such a way that its concatenate values into a string.
- **Inputs:** Array of nonnegative integers.
- **Outputs:** String.
- **Constraints:**
- **Assumptions:**
- **Edge cases:**

Design the solution

This problem requires two main steps: Sorting the array numbers to create the largest possible integer and creating the string itself. In order to sort the integers, a modified bubble sort algorithm was used, in which rather than directly comparing one integer to the next, both integers are combined both ways to determine which order creates the largest number (in order to avoid unnecessary string conversion, the integers are combined directly as integers by multiplying the first integer by the second one's number of digits). Once the array is sorted, one last iteration concatenates each integer

into a string in order to return this final string as the largest possible number.

Write pseudocode

```

1 n ← SizeOf(Vector)
2 for v ← 0 to n - 2 do
3     for i ← 0 to n - v - 2 do
4         digits1 ← round down log10(Vector[i]) + 1
5         digits2 ← round down log10(Vector[i + 1]) + 1
6         num1 ← Vector[i] * 10digits2 + Vector[i + 1]
7         num2 ← Vector[i + 1] * 10digits1 + Vector[i]
8         if num1 < num2 then
9             swap(Vector[i], Vector[i + 1])
10        end if
11    end for
12 end for
13 Order ← ""
14 for i ← 0 to n - 1 do
15     append Vector[i] to Order
16 end for
17 return Order

```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Vector})$	1
Initialize $v \leftarrow 0$	1
Check $v < n - 2$	$n - 1$
Increment v	$n - 2$
Initialize $i \leftarrow 0$	$n - 2$
Check $i < n - v - 2$	$\frac{(n-1)n}{2}$
Increment i	$\frac{(n-2)(n-1)}{2}$
Compute $\text{digits1} \leftarrow \log_{10}(\text{Vector}[i])$	$\frac{(n-2)(n-1)}{2}$
Round up previous computation	$\frac{(n-2)(n-1)}{2}$
Compute $\text{digits2} \leftarrow \log_{10}(\text{Vector}[i + 1])$	$\frac{(n-2)(n-1)}{2}$
Round up previous computation	$\frac{(n-2)(n-1)}{2}$
Compute $\text{num1} \leftarrow \text{Vector}[i] * 10^{\text{digits1}} + \text{Vector}[i + 1]$	$\frac{(n-2)(n-1)}{2}$
Compute $\text{num2} \leftarrow \text{Vector}[i + 1] * 10^{\text{digits2}} + \text{Vector}[i]$	$\frac{(n-2)(n-1)}{2}$
Check $\text{num1} < \text{num2}$	$\frac{(n-2)(n-1)}{2}$
Swap $\text{Vector}[i]$ and $\text{Vector}[i + 1]$	$\frac{(n-2)(n-1)}{2}$
Order ← ""	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Append $\text{Vector}[i]$ to Order	$n - 1$
Return Order	1

Time complexity: $O(n^2)$

Implement in C++

```
// P3
#pragma region
vector<int> case3_1 = {16,2,40,9,10};
string ans3_1 = "94021610";
assert(largestNumber(case3_1) == ans3_1);

vector<int> case3_2 = {};
string ans3_2 = "";
assert(largestNumber(case3_1) == ans3_1);

cout << "Problem 3 Test Cases: Passed" << endl;
#pragma endregion
```

```
scopellie@DESKTOP-0PMAWRFH:~/EECE7205$ ./home/scopellie/EECE7205/Assignment01_Scopelli
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

Problem 4

Description: Anagrams contain the same characters with the same frequencies but in a different order. Grouping them is useful in text indexing and search.

Question: Given a vector of strings, group the anagrams together. The order of groups does not matter, but words within each group should retain their original input order.

Understand the problem

- **Goal:** Group anagrams together.
- **Inputs:** Array of strings.
- **Outputs:** Array of vectors containing grouped anagrams.
- **Constraints:**
- **Assumptions:** String case will be considered as different characters.
- **Edge cases:** Empty input array.

Design the solution

Hello.

Write pseudocode

```

1  n ← SizeOf(Words)
2  g ← map [] // [sorted characters , [words]]
3  groups ← [] // [[words]]
4  keys ← [] // [sorted characters]
5
6  for i ← 0 to n - 1 do
7      key ← words[i]
8      sort(key)
9
10     if g contains key then
11         append words[i] to g
12     else
13         g[key] ← [words[i]]
14         append key to keys
15     end if
16 end for
17
18 for each key in keys do
19     append g[key] to groups
20 end for
21 return groups

```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Words})$	1
$g \leftarrow []$	1
$groups \leftarrow []$	1
$keys \leftarrow []$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
$key \leftarrow \text{words}[i]$	$n - 1$
Sort(key)	$(n - 1) * O(n \log(n))$
Check key in g	$\log(n)$
Append $\text{Words}[i]$ to g	$n - 1$
Else statement	0
$g[key] = [\text{words}[i]]$	$n - 1$
Append key to $keys$	$n - 1$
Initialize each $keys$	1
Iterate over $keys$	n
Increment key	$n - 1$
Append $g[key]$ to $groups$	$n - 1$
Return $groups$	1

Time complexity: $O(n^2 \log(n))$

Implement in C++

```
// P4
#pragma region
vector<string> case4_1 = {"eat","tea","tan","ate","nat","bat"};
vector<vector<string>> ans4_1 = {{{"eat","tea","ate"}, {"tan","nat"}, {"bat"}}};
assert(groupAnagrams(case4_1) == ans4_1);

vector<string> case4_2 = {""};
vector<vector<string>> ans4_2 = {{{""}}};
assert(groupAnagrams(case4_2) == ans4_2);

vector<string> case4_3 = {"abc", "def", "ghi"};
vector<vector<string>> ans4_3 = {{{"abc"}, {"def"}, {"ghi"}}};
assert(groupAnagrams(case4_3) == ans4_3);

cout << "Problem 4 Test Cases: Passed" << endl;
```

```
● (.venv) scopellie@DESKTOP-0PMDARFH:~/EECE7205/Assignment01_Scopellie
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

Problem 5

Description: Calendar software must determine whether a person or resource has been assigned to overlapping meetings.

Question: Given meeting intervals [start, end], determine whether one person can attend every meeting. Meetings that begin exactly when other ends are allowed.

Understand the problem

- **Goal:** Determine whether a person has any overlapping meeting intervals.
- **Inputs:** Array of intervals [start, end].
- **Outputs:** Boolean value indicating if the person can attend all meeting intervals (no overlap).
- **Constraints:**
- **Assumptions:** Touching intervals do not count as overlap.
- **Edge cases:** Single interval or empty input array.

Design the solution

Similar to the first problem, in order to identify any overlap between time intervals, the input array will be sorted by the start times of each interval. Once all intervals are properly sorted, we can iterate over every pair of intervals and check if any intervals overlap. In case an overlap is identified, the function immediately returns a value of False, indicating that the person can't attend all meeting intervals due to there being at least one interval. In case the iteration goes through all intervals and no overlap is identified, the function returns a value of True.

Write pseudocode

```

1  n ← SizeOf(Intervals)
2
3  if n <= 1 then
4      return true
5  end if
6
7  for Interval ← 0 to n - 2 do
8      for i ← 0 to n - Interval - 2 do
9          if Intervals[i].start > Intervals[i + 1].start then
10             swap(Intervals[i], Intervals[i + 1])
11         end if
12     end for
13 end for
14
15 for i ← 1 to n - 1 do
16     if Intervals[i].start < Intervals[i - 1].end then
17         return false
18     end if
19 end for
20
21 return true

```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Intervals})$	1
Check $n \leq 1$	1
return true	1
Initialize $\text{Interval} \leftarrow 0$	1
Check $\text{Interval} < n - 2$	$n - 1$
Increment Interval	$n - 2$
Initialize $i \leftarrow 0$	$n - 2$
Check $i < n - \text{Interval} - 2$	$\frac{(n-1)n}{2}$
Increment i	$\frac{(n-2)(n-1)}{2}$
Check $\text{Intervals}[i].\text{start} > \text{Intervals}[i + 1].\text{start}$	$\frac{(n-2)(n-1)}{2}$
Swap $\text{Intervals}[i].\text{start}$ and $\text{Intervals}[i + 1].\text{start}$	$\frac{(n-2)(n-1)}{2}$
Initialize $i \leftarrow 1$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Check $\text{Intervals}[i].\text{start} < \text{Intervals}[i - 1].\text{end}$	$n - 1$
Return false	1
Return true	1

Time complexity (Best case): $3 \rightarrow O(1)$

Time complexity (Worst case): $2n^2 + n + 1 \rightarrow O(n^2)$

Implement in C++

```
// P5
#pragma region
vector<vector<int>> case5_1 = {{1,3}, {15,18}, {8,10}, {2,6}};
bool ans5_1 = false;
assert(checkAvailability(case5_1) == ans5_1);

vector<vector<int>> case5_2 = {{1,7}, {7,10}, {10,12}};
bool ans5_2 = true;
assert(checkAvailability(case5_2) == ans5_2);

vector<vector<int>> case5_3 = {{1,5}};
bool ans5_3 = true;
assert(checkAvailability(case5_3) == ans5_3);

cout << "Problem 5 Test Cases: Passed" << endl;
```

```
(.venv) scopellie@DESKTOP-0PQWAFH:~/EECE7205$ /home/scopellie/EECE7205/Assignment01_Scopellie
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

Problem 6

Description: Navigation, robotics, and spatial systems often need to identify the nearest locations or sensor readings.

Question: Given points (x, y) and an integer k , return the k points closest to the origin. Compare a sorting solution with a heap-based solution.

Understand the problem

- **Goal:** Return the k nearest points (x, y) to the origin $(0, 0)$.
- **Inputs:** Array of points (x, y) .
- **Outputs:** List of k closest points (x, y) .
- **Constraints:**
- **Assumptions:** The origin is $(0, 0)$.
- **Edge cases:**

Design the solution

In order to determine the closest points from the origin, the distances from the origin to each point need to be calculated. Having calculated these distances, the points can be sorted through said distance, ensuring the closest points are set in order in the original array. From that point, we simply have to iterate through the first k points in this list and return these points in a separate list.

Write pseudocode

```
1 n ← SizeOf(Points)
2 pdist ← map [] // [index, distance]
3
4 for i ← 0 to n - 1 do
5     distance ← sqrt(x2 + y2)
6     append [index, distance] to pdist
7 end for
8
9 for i ← 0 to n - 2 do
10     for j ← 0 to n - i - 2 do
11         if pdist[j].distance > pdist[j + 1].distance then
12             swap(pdist[j], pdist[j + 1])
13         end if
14     end for
15 end for
16
17 result ← []
18 for i ← 1 to k - 1 do
19     append points[pdist[i].index] to result
20 end for
21
22 return result
```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Points})$	1
$\text{pdist} \leftarrow []$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Compute $\text{distance} \leftarrow \sqrt{x^2 + y^2}$	$n - 1$
Append $[\text{index}, \text{distance}]$ to pdist	$n - 1$
$n \leftarrow \text{SizeOf}(\text{Vector})$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 2$	$n - 1$
Increment i	$n - 2$
Initialize $j \leftarrow 0$	$n - 2$
Check $j < n - i - 2$	$\frac{(n-1)n}{2}$
Increment j	$\frac{(n-2)(n-1)}{2}$
Check $\text{pdist}[j].\text{distance} > \text{pdist}[j+1].\text{distance}$	$\frac{(n-2)(n-1)}{2}$
Swap $\text{pdist}[j]$ and $\text{pdist}[j+1]$	$\frac{(n-2)(n-1)}{2}$
Result $\leftarrow []$	1
Initialize $i \leftarrow 0$	1
Check $i < k - 1$	n
Increment i	$n - 1$
Append $\text{Points}[\text{pdist}[i].\text{index}]$ to Result	$n - 1$
Return Result	1

Time complexity: $2n^2 + 5n + 1 \rightarrow O(n^2)$

Implement in C++

```
// P6
#pragma region
vector<vector<int>> case6_1 = {{1,3}, {2,2}, {4,2}};
int k6_1 = 2;
vector<vector<int>> ans6_1 = {{2,2},{1,3}};
assert(nearestPoints(case6_1, k6_1) == ans6_1);

vector<vector<int>> case6_2 = {{1,1}};
int k6_2 = 1;
vector<vector<int>> ans6_2 = {{1,1}};
assert(nearestPoints(case6_2, k6_2) == ans6_2);

vector<vector<int>> case6_3 = {{0,1}, {1,0}, {1,-1}};
int k6_3 = 2;
vector<vector<int>> ans6_3 = {{0,1},{1,0}};
assert(nearestPoints(case6_3, k6_3) == ans6_3);

cout << "Problem 6 Test Cases: Passed" << endl;
```

```
• (.venv) scopellie@DESKTOP-6WDARFH:~/EECE7205$ /home/scopellie/EECE7205/Assignment01_Scopelli
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

Problem 7

Description: Monitoring and analytics systems often rank events, error codes, or queries by frequency.

Question: Given an integer array and an integer k , return the k most frequent values. If frequencies tie, return the smaller value first.

Understand the problem

- **Goal:** Return the k most frequent values in an array.
- **Inputs:** Array of integers, k .
- **Outputs:** List of k most frequent values.
- **Constraints:** In case of a tie, the smaller value gets returned first.
- **Assumptions:**
- **Edge cases:**

Design the solution

For this problem, the first step is to count the frequency of every value in the input array through a map data structure where the array values act as the keys, and the frequency as the values. Having a value counter for all the elements in the array, we must return only the k most frequent ones, which can be done by iterating through the map, and taking out the largest frequency k times, finally returning the list with these removed values.

Write pseudocode

```
1 n ← SizeOf(I)
2 counters ← map [] // [number, frequency]
3
4 for i ← 0 to n - 1 do
5     if counters contains I[i] then
6         counters[I[i]] ← counters[I[i]] + 1
7     else
8         counters[I[i]] ← 1
9     end if
10 end for
11
```

```

12 values ← []
13 for i ← 0 to k - 1 do
14     max ← [] // [number, frequency]
15     for each counter in counters do
16         if max.frequency < counter.frequency then
17             max ← counter
18         end if
19     end for
20     remove max.number from counters
21     append max.number to values
22 end for
23
24 return values

```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(I)$	1
$\text{counters} \leftarrow []$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Check $\text{counters} \text{ contains } I[i]$	$n - 1$
Increment $\text{counters}[I[i]]$	$n - 1$
Else statement	0
$\text{counters}[I[i]] \leftarrow 1$	$n - 1$
$\text{values} \leftarrow []$	1
Initialize $i \leftarrow 0$	1
Check $i < k - 1$	n
Increment i	$n - 1$
$\text{max} \leftarrow []$	$n - 1$
Initialize each counters	$n - 1$
Iterate over counters	$\frac{(n-1)n}{2}$
Increment counter	$\frac{(n-2)(n-1)}{2}$
Check $\text{max.frequency} < \text{counter.frequency}$	$\frac{(n-2)(n-1)}{2}$
$\text{max} \leftarrow \text{counter}$	$\frac{(n-2)(n-1)}{2}$
Remove max.number from counters	$n - 1$
Append max.number to values	$n - 1$
Return values	1

Time complexity: $2n^2 + 5n + 1 \rightarrow O(n^2)$

Implement in C++

```
// P7
#pragma region
vector<int> case7_1 = {1,4,2,2,1,1,3};
int k7_1 = 2;
vector<int> ans7_1 = {1,2};
assert(frequentValues(case7_1, k7_1) == ans7_1);

vector<int> case7_2 = {1,1,2,2,3};
int k7_2 = 2;
vector<int> ans7_2 = {1,2};
assert(frequentValues(case7_2, k7_2) == ans7_2);

vector<int> case7_3 = {1};
int k7_3 = 1;
vector<int> ans7_3 = {1};
assert(frequentValues(case7_3, k7_3) == ans7_3);

cout << "Problem 7 Test Cases: Passed" << endl;
```

```
(.venv) scopellie@DESKTOP-0PQW8FH:~/EECE7205$ ./home/scopellie/EECE7205/Assignment01_Scopellie
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

Problem 8

Description: Distributed systems may produce multiple individually sorted streams that must be combined into one ordered result.

Question: Given k sorted integer arrays, merge them into one sorted array. Design a solution that does not repeatedly sort the complete output.

Understand the problem

- **Goal:** Sort values from multiple sorted arrays into a single array.
- **Inputs:** Array of sorted arrays.
- **Outputs:** Single sorted array.
- **Constraints:** The solution must not repeatedly sort the complete output.
- **Assumptions:** Arrays can have different sizes.
- **Edge cases:**

Design the solution

In order to sort integers from multiple sorted arrays into a single array without repeatedly sorting the merged output, we can take advantage of these arrays already being sorted, simply iterating over the smallest values in each array and maintaining a pointer to indicate which value from each array is still missing from the sorted array. For the pointer, we need to determine the size of each array, and essentially create an index for each one, which can be done through two arrays. Once these indexes are initialized and the upper limit (array sizes) is established, we iterate through the combined elements of the different arrays, with each iteration determining the "minimum" value to add next to our final sorted array.

Write pseudocode

```
1 n ← SizeOf(M)
2 sorted ← []
3 sizes ← []
4
5 for i ← 0 to n - 1 do
6     append SizeOf(M[i]) to sizes
7 end for
8
9 pointers ← n * [0] // Initialize an index for every array
10 while sizes != pointers do
11     min ← null
12     index ← 0
13     for i ← 0 to n - 1 do
14         if pointers[i] < sizes[i] then
15             if min = null or min > M[i][pointers[i]] then
16                 min ← M[i][pointers[i]]
17                 index ← i
18             end if
19         end if
20     end for
21     append min to sorted
22     pointers[index] ← pointers[index] + 1
23 end while
24
25 return sorted
```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(M)$	1
$\text{sorted} \leftarrow []$	1
$\text{sizes} \leftarrow []$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
$n_i \leftarrow \text{SizeOf}(M[i])$	$n - 1$
Append n_i to sizes	$n - 1$
$\text{pointers} \leftarrow n * [0]$	1
Initialize while loop	1
Check $\text{sizes} \neq \text{pointers}$	n
Increment pointers	$n - 1$
$\text{min} \leftarrow \text{null}$	$n - 1$
$\text{index} \leftarrow 0$	$n - 1$
Initialize $i \leftarrow 0$	$n - 1$
Check $i < n - 1$	$\frac{(n-1)n}{2}$
Increment i	$\frac{(n-2)(n-1)}{2}$
Check $\text{pointers}[i] < \text{sizes}[i]$	$\frac{(n-2)(n-1)}{2}$
Check $\text{min} = \text{null}$ or $\text{min} > M[i][\text{pointers}[i]]$	$\frac{(n-2)(n-1)}{2}$
$\text{min} \leftarrow M[i][\text{pointers}[i]]$	$\frac{(n-2)(n-1)}{2}$
$\text{index} \leftarrow i$	$\frac{(n-2)(n-1)}{2}$
Append min to sorted	$n - 1$
Increment $\text{pointers}[\text{index}]$	$n - 1$
Return sorted	1

Time complexity: $3n^2 + 3n + 3 \rightarrow O(n^2)$

Implement in C++

```
// P8
#pragma region
vector<vector<int>> case8_1 = {{1,2,4},{2,3},{4,5,6}};
vector<int> ans8_1 = {1,2,2,3,4,4,5,6};
assert(mergeSorted(case8_1) == ans8_1);

vector<vector<int>> case8_2 = {{1,2,3},{}};
vector<int> ans8_2 = {1,2,3};
assert(mergeSorted(case8_2) == ans8_2);

vector<vector<int>> case8_3 = {{1,2,3},{4,5,6}};
vector<int> ans8_3 = {1,2,3,4,5,6};
assert(mergeSorted(case8_3) == ans8_3);

cout << "Problem 8 Test Cases: Passed" << endl;
```

```
• (.venv) scopellie@DESKTOP-GMDARFH:~/EECE7205$ /home/scopellie/EECE7205/Assignment01_Scopelli
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

Problem 9

Description: Ranking systems often need one order statistic without paying the cost of fully sorting a large dataset.

Question: Given an unsorted integer array and k , return the k th-largest element. Duplicate values count as separate positions.

Understand the problem

- **Goal:** Return the k largest value in an unsorted array.
- **Inputs:** Array of numbers, k .
- **Outputs:** K largest integer.
- **Constraints:** Ideally the solution shouldn't have to sort the entire array.
- **Assumptions:**
- **Edge cases:**

Design the solution

Having an unsorted array and requiring the k -th largest integer, we can utilize a partial sort up until we have the required integer, without sorting the rest of the array. In order to do this, we simply use a regular sorting algorithm, but limit the iterations to the input k and return this last sorted integer.

Write pseudocode

```
1 n ← SizeOf(A)
2 if k > n then
3     return null
4 end if
5
6 for i ← 0 to n - 2 do
7     for j ← 0 to n - i - 2 do
8         if A[j] > A[j + 1] then
9             swap(A[j], A[j + 1])
10        end if
11    end for
12    if i = k - 1 then
```

```

13         return A[n - k]
14     end if
15 end for
16
17 return A[n - k]

```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(A)$	1
Check $k > n$	1
Return <i>null</i>	1
Initialize $i \leftarrow 0$	1
Check $i < n - 2$	$n - 1$
Increment i	$n - 2$
Initialize $j \leftarrow 0$	$n - 2$
Check $j < n - i - 2$	$\frac{(n-1)n}{2}$
Increment j	$\frac{(n-2)(n-1)}{2}$
Check $A[j] > A[j + 1]$	$\frac{(n-2)(n-1)}{2}$
Swap $A[j], A[j + 1]$	$\frac{(n-2)(n-1)}{2}$
Check $i == k - 1$	$n - 2$
Return $A[n - k]$	$n - 2$
Return $A[n - k]$	1

Time complexity (Best case): $3 \rightarrow O(1)$

Time complexity (Worst case): $2n^2 + n - 1 \rightarrow O(n^2)$

Implement in C++

```

// P9
#pragma region
vector<int> case9_1 = {6,5,4,3,2,1};
int k9_1 = 2;
int ans9_1 = 5;
assert(kLargestElement(case9_1, k9_1) == ans9_1);

vector<int> case9_2 = {};
int k9_2 = 1;
int ans9_2 = NULL;
assert(kLargestElement(case9_2, k9_2) == ans9_2);

vector<int> case9_3 = {3, 2, 3, 1, 2, 4, 5, 5, 6};
int k9_3 = 6;
int ans9_3 = 3;
assert(kLargestElement(case9_3, k9_3) == ans9_3);

cout << "Problem 9 Test Cases: Passed" << endl;

```

```
• (.venv) scopellie@DESKTOP-GMDARFH:~/EECE7205$ /home/scopellie/EECE7205/Assignment01_Scopelli
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

Problem 10

Description: An inversion measures how far an array is from sorted order and is used in ranking analysis and sequence comparison.

Question: Count pairs (i, j) for which $i < j$ and $\text{values}[i] > \text{values}[j]$. Develop an efficient solution by modifying Merge Sort.

Understand the problem

- **Goal:** Determine number of inversions required to sort an array.
- **Inputs:** Array of numbers.
- **Outputs:** Number of inversions.
- **Constraints:**
- **Assumptions:**
- **Edge cases:**

Design the solution

To determine the required inversions in an unsorted array, an iteration over all pairs of numbers that identifies any pair of integers where the larger value has a smaller index directly provides these inversions. Doing these iterations as recursive to go through every pair of integers gives us our "brute-force" solution similar to the basic sorting algorithms.

Write pseudocode

```
1 n ← SizeOf(A)
2 count ← 0
3
4 for i ← 0 to n - 2 do
5     for j ← i + 1 to n - 1 do
6         if A[i] > A[j] then
7             count ← count + 1
8         end if
9     end for
10 end for
11
12 return count
```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(A)$	1
$\text{count} \leftarrow 0$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 2$	$n - 1$
Increment i	$n - 2$
Initialize $j \leftarrow i + 1$	$n - 2$
Check $j < n - 1$	$\frac{(n-1)n}{2}$
Increment j	$\frac{(n-2)(n-1)}{2}$
Check $A[i] > A[j]$	$\frac{(n-2)(n-1)}{2}$
Increment count	$\frac{(n-2)(n-1)}{2}$
Return count	1

Time complexity: $2n^2 - 2n + 5 \rightarrow O(n^2)$

Implement in C++

```
// P10
#pragma region
vector<int> case10_1 = {2,4,1,3,5};
int ans10_1 = 3;
assert(countInversions(case10_1) == ans10_1);

vector<int> case10_2 = {};
int ans10_2 = 0;
assert(countInversions(case10_2) == ans10_2);

vector<int> case10_3 = {4,3,2,1};
int ans10_3 = 6;
assert(countInversions(case10_3) == ans10_3);

cout << "Problem 10 Test Cases: Passed" << endl;
```

```
● (.venv) scopellie@DESKTOP-0MDARFH:~/EECE7205$ ./home/scopellie/EECE7205/Assignment01_Scopellie
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```